Informe de laboratorio 08

Tema: Métodos Heurísticos: Problemas NP y Búsqueda Aleatoria (Monte Carlo)

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
08	Métodos Heurísticos: Problemas NP y Búsqueda Aleatoria (Monte Carlo)	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	05/11/2026	09/11/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Implementa fuerza bruta para un TSP de 10 ciudades generadas aleatoriamente, y mide cuánto tiempo tarda en encontrar la solución óptima.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
#include <chrono>
#include <iomanip>
#include "tsp_comun.h"
using namespace std;

int main() {
    const int N = 10;
    const unsigned SEMILLA = 2026;

    vector<Ciudad> ciudades = generarCiudades(N, SEMILLA);
```

```
    vector<vector<int>> dist = matrizDistancias(ciudades);

    cout << "Ciudades generadas (id: x, y):\n";
    for (int i = 0; i < N; i++)
        cout << "  " << i << ": (" << ciudades[i].x << ", " << ciudades[i].y << ")\n";

    // La ciudad 0 es el origen fijo; se permutan solo las ciudades 1..N-1.
    vector<int> perm;
    for (int i = 1; i < N; i++) perm.push_back(i);

    int mejorCosto = INT_MAX;
    vector<int> mejorRuta;
    long long evaluadas = 0;

    auto inicio = chrono::steady_clock::now();

    do {
        if (perm.front() > perm.back()) continue;

        evaluadas++;
        int costo = costoRuta(dist, perm);
        if (costo < mejorCosto) {
            mejorCosto = costo;
            mejorRuta = perm;
        }
    } while (next_permutation(perm.begin(), perm.end()));

    auto fin = chrono::steady_clock::now();
    double ms = chrono::duration<double, milli>(fin - inicio).count();

    cout << "\nRuta optima (fuerza bruta): 0 -> ";
    for (int c : mejorRuta) cout << c << " -> ";
    cout << "0\n";
    cout << "Costo optimo: " << mejorCosto << "\n";
    cout << "Rutas distintas evaluadas: " << evaluadas << "\n";
    cout << fixed << setprecision(3);
    cout << "Tiempo de ejecucion: " << ms << " ms\n";

    return 0;
}
```

- Genera 10 ciudades con coordenadas reproducibles, calcula sus distancias y evalúa rutas que empiezan y terminan en la ciudad 0. Como la distancia es simétrica, una ruta y su recorrido inverso cuestan lo mismo; por eso descarta una de cada pareja. Al final informa la ruta más corta, su costo y el tiempo empleado. En esencia, busca la mejor ruta examinando todas las alternativas posibles.

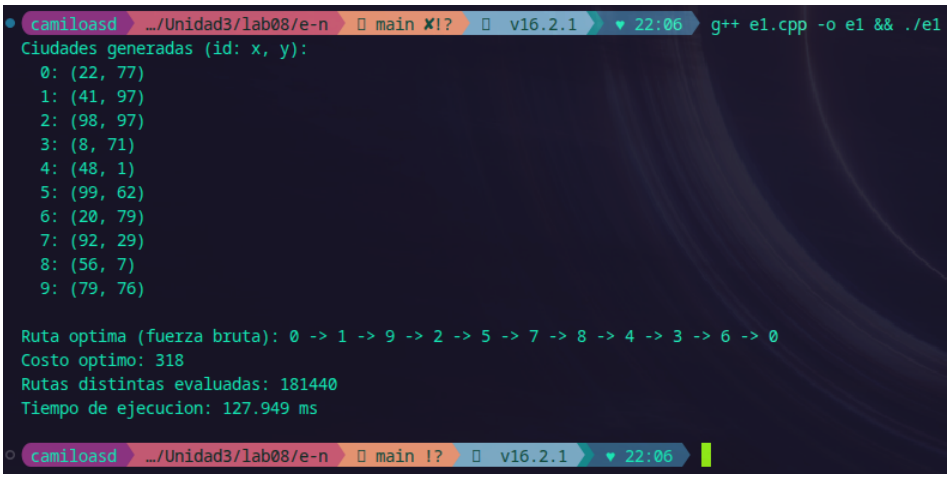


Figura 1: Ejecución del ejercicio 1

1.2. Ejercicio 2

Implementa búsqueda aleatoria (Monte Carlo) para el mismo TSP de 10 ciudades del ejercicio anterior, con distintos números de iteraciones (100, 1 000, 10 000), y compara qué tan cerca queda del óptimo real en cada caso.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
#include <random>
#include <iomanip>
#include <numeric>
#include <sstream>
#include "tsp_comun.h"
using namespace std;

// Optimo real por fuerza bruta (misma logica del ejercicio 1).
int optimoFuerzaBruta(const vector<vector<int>> &dist, int N) {
    vector<int> perm;
    for (int i = 1; i < N; i++) perm.push_back(i);
    int mejor = INT_MAX;
    do {
        if (perm.front() > perm.back()) continue;
        mejor = min(mejor, costoRuta(dist, perm));
    } while (next_permutation(perm.begin(), perm.end()));
    return mejor;
}

// Una corrida de Monte Carlo: devuelve el mejor costo visto tras "iteraciones" rutas al azar.
int monteCarlo(const vector<vector<int>> &dist, int N, int iteraciones, mt19937 &rng) {
    vector<int> ruta;
    for (int i = 1; i < N; i++) ruta.push_back(i);
    int mejor = INT_MAX;
    for (int it = 0; it < iteraciones; it++) {
```

```cpp
        shuffle(ruta.begin(), ruta.end(), rng);
        mejor = min(mejor, costoRuta(dist, ruta));
    }
    return mejor;
}

int main() {
    const int N = 10;
    const unsigned SEMILLA = 2026;          // misma instancia que el ejercicio 1
    const int REPETICIONES = 100;

    vector<vector<int>> dist = matrizDistancias(generarCiudades(N, SEMILLA));
    int optimo = optimoFuerzaBruta(dist, N);
    cout << "Optimo real (fuerza bruta): " << optimo << "\n\n";

    mt19937 rng(12345);
    vector<int> configuraciones = {100, 1000, 10000};

    cout << "Cada fila resume " << REPETICIONES << " corridas independientes de Monte
    ↪  Carlo.\n\n";
    cout << left << setw(12) << "Iteraciones" << setw(14) << "1ra corrida"
         << setw(12) << "Mejor" << setw(12) << "Peor" << setw(12) << "Promedio"
         << setw(14) << "Brecha prom." << "Veces optimo\n";
    cout << string(88, '-') << "\n";

    for (int k : configuraciones) {
        vector<int> resultados;
        for (int r = 0; r < REPETICIONES; r++)
            resultados.push_back(monteCarlo(dist, N, k, rng));

        int mejor = *min_element(resultados.begin(), resultados.end());
        int peor = *max_element(resultados.begin(), resultados.end());
        double prom = accumulate(resultados.begin(), resultados.end(), 0.0) / REPETICIONES;
        double brecha = (prom - optimo) * 100.0 / optimo;
        int aciertos = count(resultados.begin(), resultados.end(), optimo);

        cout << left << setw(12) << k << setw(14) << resultados[0]
             << setw(12) << mejor << setw(12) << peor
             << setw(12) << fixed << setprecision(1) << prom
             << setw(14) << ([&]{ ostringstream o; o << fixed << setprecision(1) << brecha
             ↪  << "%"; return o.str(); }())
             << aciertos << "/" << REPETICIONES << "\n";
    }
    return 0;
}
```

- Primero calcula el óptimo exacto para tener una referencia. Después mezcla las ciudades al azar y conserva la ruta de menor costo en cada corrida. Repite el experimento con distintas cantidades de intentos y resume los resultados con el mejor, peor y promedio, además de cuántas veces encontró el óptimo. Más intentos suelen mejorar la respuesta, pero el azar no garantiza acertar.



Figura 2: Ejecución del ejercicio 2

1.3. Ejercicio 3

Modela una versión simplificada del problema de programación de líneas de producción (por ejemplo, 6 tareas y 2 líneas) y resuélvela con búsqueda aleatoria, minimizando el tiempo total de finalización.

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>
#include <random>
#include <iomanip>
using namespace std;

const int NUM_TAREAS = 6;
const int NUM_LINEAS = 2;

int makespan(const vector<int> &duracion, const vector<int> &asignacion) {
    vector<int> carga(NUM_LINEAS, 0);
    for (int i = 0; i < NUM_TAREAS; i++)
        carga[asignacion[i]] += duracion[i];
    return *max_element(carga.begin(), carga.end());
}

void imprimirAsignacion(const vector<int> &duracion, const vector<int> &asignacion) {
    vector<int> carga(NUM_LINEAS, 0);
    for (int l = 0; l < NUM_LINEAS; l++) {
        cout << "  Linea " << l + 1 << ": ";
        for (int i = 0; i < NUM_TAREAS; i++)
            if (asignacion[i] == l) {
                cout << "T" << i + 1 << "(" << duracion[i] << "h) ";
                carga[l] += duracion[i];
            }
        cout << " -> termina a las " << carga[l] << " h\n";
    }
}

int main() {
    vector<int> duracion = {5, 3, 8, 2, 7, 4}; // horas de cada tarea
    int total = 0;
    for (int d : duracion) total += d;

    cout << "Tareas (horas): ";
```

```
    for (int i = 0; i < NUM_TAREAS; i++) cout << "T" << i + 1 << "=" << duracion[i] << " ";
    cout << "\nSuma total = " << total << " h  =>  cota inferior del makespan = "
         << (total + NUM_LINEAS - 1) / NUM_LINEAS << " h\n\n";

    // Busqueda aleatoria
    mt19937 rng(2026);
    uniform_int_distribution<int> lineaAlAzar(0, NUM_LINEAS - 1);

    const int ITERACIONES = 1000;
    int mejorMC = INT_MAX;
    vector<int> mejorAsignacion;

    cout << "Busqueda aleatoria (" << ITERACIONES << " iteraciones). Mejoras encontradas:\n";
    for (int it = 1; it <= ITERACIONES; it++) {
        vector<int> asignacion(NUM_TAREAS);
        for (int i = 0; i < NUM_TAREAS; i++)
            asignacion[i] = lineaAlAzar(rng);      // cada tarea va a una linea al azar

        int ms = makespan(duracion, asignacion);
        if (ms < mejorMC) {
            mejorMC = ms;
            mejorAsignacion = asignacion;
            cout << "  iteracion " << setw(4) << it << ": makespan = " << ms << " h\n";
        }
    }
    cout << "\nMejor asignacion encontrada (makespan " << mejorMC << " h):\n";
    imprimirAsignacion(duracion, mejorAsignacion);

        // Verificacion con fuerza bruta
    int optimo = INT_MAX;
    long long total_asig = 1;
    for (int i = 0; i < NUM_TAREAS; i++) total_asig *= NUM_LINEAS;
    for (long long codigo = 0; codigo < total_asig; codigo++) {
        vector<int> asignacion(NUM_TAREAS);
        long long c = codigo;
        for (int i = 0; i < NUM_TAREAS; i++) { asignacion[i] = c % NUM_LINEAS; c /=
        ↪  NUM_LINEAS; }
        optimo = min(optimo, makespan(duracion, asignacion));
    }
    cout << "\nVerificacion por fuerza bruta (" << total_asig << " asignaciones): makespan
    ↪  optimo = "
         << optimo << " h\n";
    cout << (mejorMC == optimo ? "Monte Carlo encontro el optimo.\n"
                               : "Monte Carlo NO llego al optimo en esta corrida.\n");
    return 0;
}
```

- Cada asignación indica a qué línea va cada tarea. La función makespan suma las horas de cada línea y toma la carga mayor: esa línea determina cuándo termina todo el trabajo. El programa prueba asignaciones aleatorias y conserva la mejor; luego revisa las 64 posibilidades mediante fuerza bruta para confirmar el óptimo. La intuición es repartir el trabajo para que una línea no quede mucho más cargada que la otra.

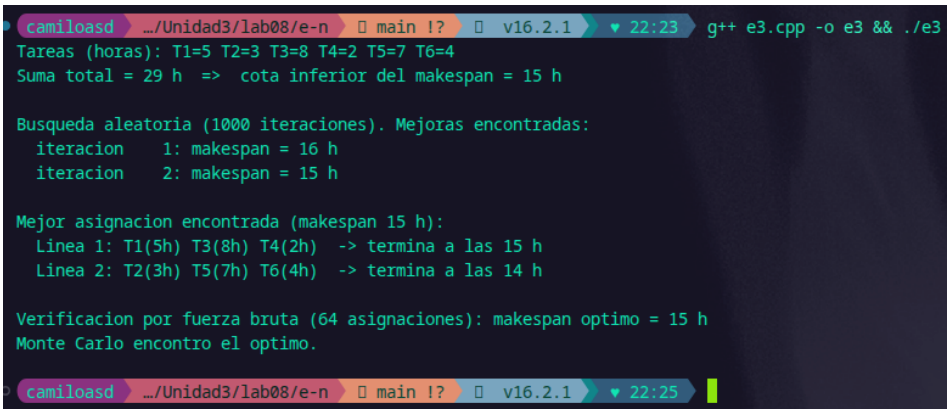


Figura 3: Ejecución del ejercicio 3

1.4. Ejercicio 4

Genera la gráfica de crecimiento del espacio de soluciones (como la Figura 3) para un problema de asignación de tareas (n! en vez de (n-1)!/2) y compárala con la del TSP.

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <string>
using namespace std;

int main() {
    const int N_MIN = 4, N_MAX = 25;

    vector<long double> fact(N_MAX + 1, 1.0L);
    for (int k = 1; k <= N_MAX; k++) fact[k] = fact[k - 1] * k;

    cout << left << setw(6) << "n" << setw(16) << "TSP (n-1)!/2" << setw(16) << "Asig. n!"
         << "Cociente (n!)/((n-1)!/2)\n";
    cout << string(64, '-') << "\n";

    vector<double> logTSP, logASIG;
    for (int n = N_MIN; n <= N_MAX; n++) {
        long double tsp = fact[n - 1] / 2.0L;
        long double asig = fact[n];
        logTSP.push_back((double)log10l(tsp));
        logASIG.push_back((double)log10l(asig));
        cout << left << setw(6) << n << setw(16) << scientific << setprecision(3) <<
        ↪  (double)tsp
             << setw(16) << (double)asig << fixed << setprecision(0) << (double)(asig / tsp)
             ↪  << "\n";
    }

    // Grafica SVG
    const int W = 900, H = 540, L = 90, R = 40, T = 60, B = 80;
    const int YMAX = 26;
```

```
    auto X = [&](int n) { return L + (double)(n - N_MIN) * (W - L - R) / (N_MAX - N_MIN); };
    auto Y = [&](double v) { return T + (H - T - B) * (1.0 - v / YMAX); };

    ofstream svg("crecimiento.svg");
    svg << "<svg xmlns='http://www.w3.org/2000/svg' viewBox='0 0 " << W << " " << H
        << "' font-family='sans-serif' font-size='13'>\n";
    svg << "<rect width='100%' height='100%' fill='white'/>\n";
    svg << "<text x='" << W / 2 << "' y='30' text-anchor='middle' font-size='18'
    ↪  font-weight='bold'>"
        << "Crecimiento del espacio de soluciones (escala logaritmica)</text>\n";

    // rejilla horizontal: cada 2 decadas
    for (int k = 0; k <= YMAX; k += 2) {
        double y = Y(k);
        svg << "<line x1='" << L << "' y1='" << y << "' x2='" << W - R << "' y2='" << y
            << "' stroke='#dddddd'/>\n";
        svg << "<text x='" << L - 8 << "' y='" << y + 4 << "' text-anchor='end'>10^" << k <<
        ↪  "</text>\n";
    }
    // marcas del eje X
    for (int n = N_MIN; n <= N_MAX; n++) {
        svg << "<text x='" << X(n) << "' y='" << H - B + 22 << "' text-anchor='middle'>" <<
        ↪  n << "</text>\n";
        svg << "<line x1='" << X(n) << "' y1='" << H - B << "' x2='" << X(n) << "' y2='" <<
        ↪  H - B + 5
            << "' stroke='black'/>\n";
    }
    svg << "<line x1='" << L << "' y1='" << H - B << "' x2='" << W - R << "' y2='" << H - B
    ↪  << "' stroke='black'/>\n";
    svg << "<line x1='" << L << "' y1='" << T << "' x2='" << L << "' y2='" << H - B << "'
    ↪  stroke='black'/>\n";
    svg << "<text x='" << (L + W - R) / 2 << "' y='" << H - 25
        << "' text-anchor='middle'>n (ciudades o tareas)</text>\n";
    svg << "<text transform='translate(25," << (T + H - B) / 2
        << ") rotate(-90)' text-anchor='middle'>Numero de soluciones posibles</text>\n";

    auto polilinea = [&](const vector<double> &v, const string &color) {
        svg << "<polyline fill='none' stroke='" << color << "' stroke-width='2.5' points='";
        for (int i = 0; i < (int)v.size(); i++) svg << X(N_MIN + i) << "," << Y(v[i]) << " ";
        svg << "'/>\n";
        for (int i = 0; i < (int)v.size(); i++)
            svg << "<circle cx='" << X(N_MIN + i) << "' cy='" << Y(v[i]) << "' r='3.5'
            ↪  fill='" << color << "'/>\n";
    };
    polilinea(logTSP, "#1f77b4");
    polilinea(logASIG, "#e8710a");

    // leyenda
    svg << "<rect x='" << L + 15 << "' y='" << T + 8 << "' width='250' height='54'
    ↪  fill='white' stroke='#999'/>\n";
    svg << "<line x1='" << L + 25 << "' y1='" << T + 26 << "' x2='" << L + 55 << "' y2='" <<
    ↪  T + 26
        << "' stroke='#1f77b4' stroke-width='3'/><text x='" << L + 62 << "' y='" << T + 30
        << "'>TSP: (n-1)!/2</text>\n";
    svg << "<line x1='" << L + 25 << "' y1='" << T + 48 << "' x2='" << L + 55 << "' y2='" <<
    ↪  T + 48
        << "' stroke='#e8710a' stroke-width='3'/><text x='" << L + 62 << "' y='" << T + 52
```

```
79          << "'>Asignacion de tareas: n!</text>\n";
80      svg << "</svg>\n";
81      svg.close();
82
83      cout << "\nGrafica guardada en crecimiento.svg\n";
84      return 0;
85  }
```

- El programa compara cuántas alternativas se deben revisar en dos problemas: para el TSP calcula $(n-1)!/2$, pues fija una ciudad de inicio y considera equivalentes una ruta y su recorrido inverso; para el ordenamiento de tareas usa $n!$. Calcula los factoriales de forma acumulativa y muestra los resultados en una tabla junto con el cociente entre ambas cantidades.
- Para visualizar el crecimiento, guarda en vectores los logaritmos decimales de cada cantidad y genera el archivo `crecimiento.svg`. La escala logarítmica permite mostrar números muy grandes en una gráfica legible: cada salto vertical representa un aumento por potencias de diez. Así se observa que ambos espacios crecen rápidamente y que enumerar todas las opciones se vuelve costoso al aumentar n.
- La fórmula $n!$ corresponde al modelo de permutar tareas indicado en la consigna. Si se tratara de asignar cada tarea a una de dos líneas, el número de asignaciones sería 2^n, no $n!$.

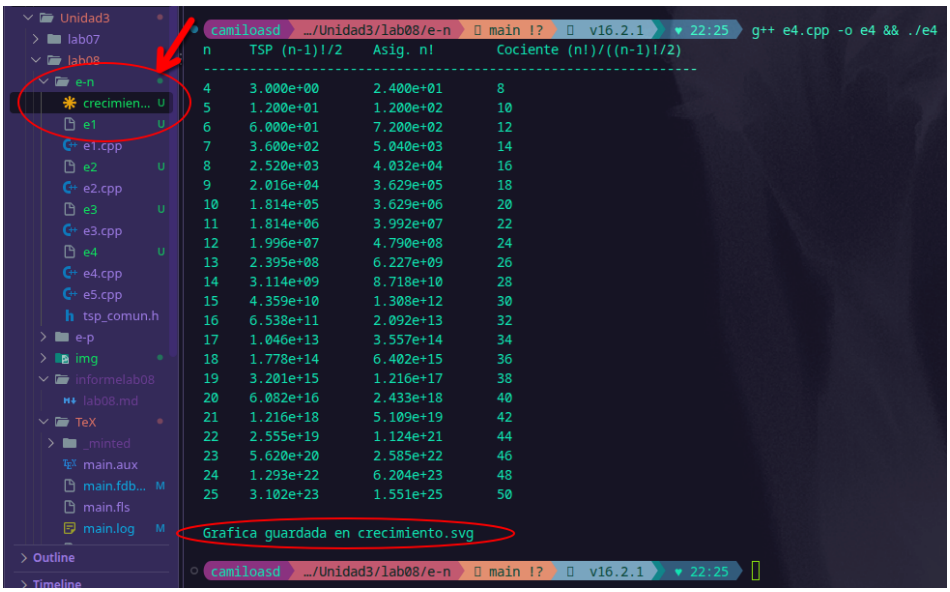


Figura 4: Ejecución del ejercicio 4

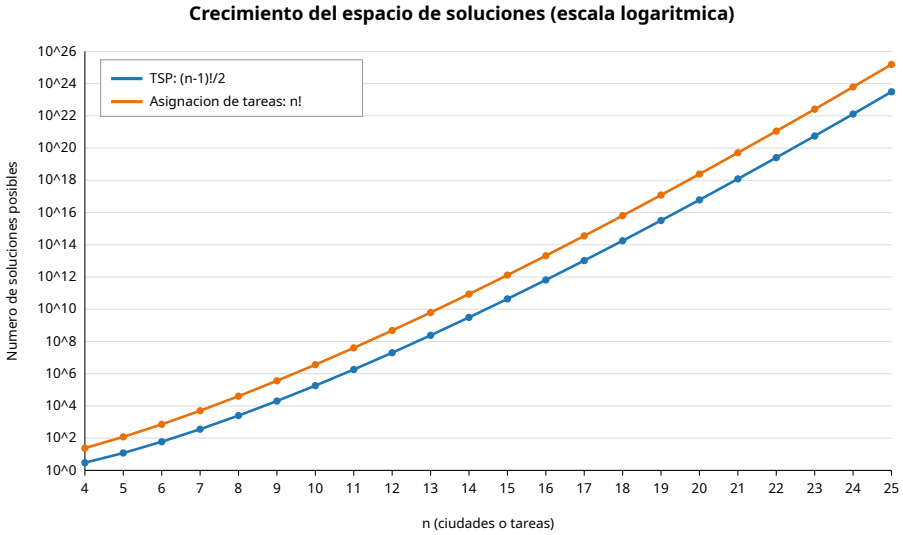


Figura 5: Gráfica de crecimiento del espacio de soluciones para TSP y asignación de tareas

1.5. Ejercicio 5

1.5.1. Caso y relación con el TSP

ORION (*On-Road Integrated Optimization and Navigation*) es el sistema de UPS que organiza las rutas diarias de reparto y recogida. Para cada conductor determina el orden de visita de sus direcciones con el objetivo de reducir el costo de operación; Holland et al. (2017) describen su aplicación en la logística de UPS.

En el modelo, los vértices representan las paradas de entrega o recogida y el depósito, que funciona como origen y destino. El costo c_{ij} de pasar de una parada i a otra j considera el tiempo y el millaje de viaje, además del servicio al cliente; por eso no equivale simplemente a distancia y puede ser asimétrico (Holland et al. 2017). El TSP clásico puede expresarse como:

$$\text{mín} \sum_{(i,j)\in A} c_{ij}x_{ij}, \qquad \sum_{j} x_{ij} = 1, \quad \sum_{i} x_{ij} = 1, \quad x_{ij} \in \{0,1\}, \tag{1}$$

donde $x_{ij} = 1$ indica que j se visita inmediatamente después de i; se agregan restricciones para evitar subtours. ORION amplía este modelo a un TSP con ventanas de tiempo (TSPTW): debe atender horarios comprometidos y mantener cierta consistencia con el orden habitual de las paradas (Holland et al. 2017).

Tabla 1: Diferencias entre el TSP clásico y el problema de rutas de UPS

Aspecto	TSP clásico	Problema de UPS
Costo	Distancia o costo de cada arco	Viaje y servicio, medidos en tiempo y millaje; puede ser asimétrico
Tiempo	Sin horarios de atención	Ventanas horarias y penalización por retrasos
Consistencia	No se considera	Se limita cuánto puede variar la posición de cada cliente respecto a la ruta base
Operación	Datos fijos, sin criterios de seguridad	Considera tráfico, seguridad en giros y cambios de la jornada

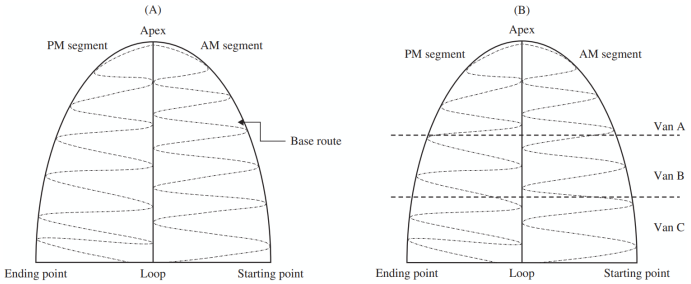


Figura 6: Ruta base de un *loop*: segmentos AM y PM unidos en un ápice y asignados a tres furgonetas (Holland et al., 2017).

1.5.2. Optimización y escala práctica

UPS probó una formulación exacta del TSPTW, pero la descartó por su tiempo de resolución. El artículo de sus autores documenta una metaheurística que combina búsqueda local con variantes de Lin–Kernighan, búsqueda adaptativa de vecindad grande, relajación lagrangiana y recocido simulado (Holland et al. 2017; Lin y Kernighan 1973; Ropke y Pisinger 2006; Lemaréchal 2001; Ingber 1993). El detalle de la implementación es secreto industrial. No se documenta que ORION use Monte Carlo puro ni programación dinámica de Held–Karp (Holland et al. 2017; Held y Karp 1962).

Las fuentes sitúan las rutas entre aproximadamente 120 y 160 paradas por conductor: se reportan unas 120 en una ruta típica, un rango de 140–160 y un promedio de 135 en otra fuente (Clevenger 2016; Holland et al. 2017; UPS 2020). El artículo de 2017 informa 55 000 conductores en Estados Unidos; en 2020 UPS informó más de 66 000 rutas en Estados Unidos, Canadá y Europa (Holland et al. 2017; UPS 2020). Los datos más recientes recogidos en la investigación corresponden a 2020–2021 (UPS 2020; Leonard 2021).

Con el depósito fijo, n paradas admiten $n!$ órdenes posibles. La tabla 2 compara esa cantidad con el ejemplo académico de cinco ciudades (cuatro paradas más el depósito). Los tiempos son cálculos ilustrativos suponiendo 10^9 evaluaciones por segundo, no mediciones de ORION.

Tabla 2: Crecimiento de las secuencias posibles de una ruta

Paradas n	Secuencias $n!$	Tiempo a 10^9 secuencias/s
4 (ejemplo de 5 ciudades)	24	Instantáneo
10	$3{,}6 \times 10^{6}$	3,6 ms
20	$2{,}4 \times 10^{18}$	≈ 77 años
120	$6{,}7 \times 10^{198}$	$\approx 10^{182}$ años
135	$2{,}7 \times 10^{230}$	$\approx 10^{214}$ años
150	$5{,}7 \times 10^{262}$	$\approx 10^{246}$ años

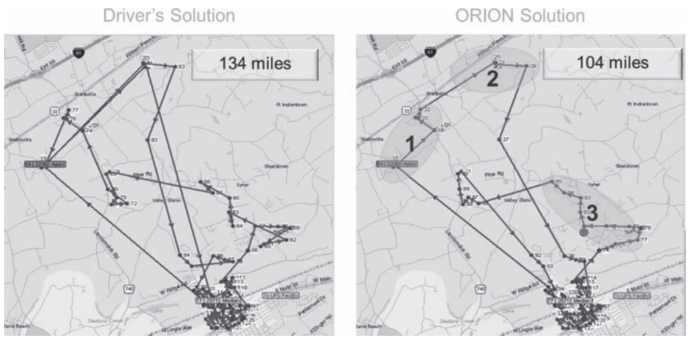


Figura 7: Comparación de la ruta del conductor y la de ORION; esta última visita dos áreas una sola vez y evita 30 millas (Holland et al., 2017).

1.5.3. Resultados y alcance

UPS atribuye a ORION ahorros anuales de aproximadamente 100 millones de millas recorridas y 10 millones de galones de combustible; la investigación también recoge una proyección cercana a 100 000 toneladas métricas de CO_2 y entre 300 y 400 millones de dólares de ahorro económico anual (Holland et al. 2017; UPS 2015; UPS 2020). Son cifras proyectadas o reportadas por UPS, no verificadas de forma independiente en este informe (Holland et al. 2017).

2. Cuestionario

1. **¿Cuál es la diferencia entre un problema de clase P y uno de clase NP?** En P están los problemas que pueden resolverse en tiempo polinómico. En NP, una solución propuesta puede comprobarse en ese tiempo. Por ejemplo, dado un recorrido, se puede verificar rápidamente que visita las ciudades requeridas y calcular su costo, aunque encontrar el recorrido óptimo pueda ser muy difícil. Se sabe que $P \subseteq NP$, pero todavía no se sabe si ambas clases son iguales.

2. **¿Por qué se dice que el TSP es un problema NP-difícil?** Porque al aumentar las ciudades, el número de rutas posibles crece factorialmente. En el laboratorio, con 10 ciudades ya se evaluaron 181 440 rutas distintas; para cientos de paradas, revisar todas las permutaciones deja de ser práctico. La versión de decisión del TSP es NP-completa y su versión de optimización es NP-difícil (Garey y Johnson 1979).

3. **¿Qué ventaja y qué desventaja principal tiene Monte Carlo frente a fuerza bruta?** Monte Carlo prueba rutas al azar y puede encontrar rápidamente una solución razonable sin recorrer todo el espacio. A cambio, no garantiza encontrar el óptimo: el resultado depende de qué alternativas salgan en los intentos y puede variar entre corridas. La fuerza bruta tarda más al crecer el problema, pero sí certifica el óptimo cuando alcanza a revisar todas las opciones.

4. **Si aumentan las iteraciones de Monte Carlo, ¿qué se puede afirmar sobre la calidad?** En una misma corrida, si se conserva siempre la mejor ruta encontrada, la calidad de ese mejor resultado no empeora y suele mejorar al probar más alternativas. También aumenta la posibilidad de acercarse al óptimo. Sin embargo, más iteraciones no garantizan encontrarlo ni permiten afirmar que una corrida independiente será mejor que otra.

5. **¿Por qué Monte Carlo es solo un primer intento ante problemas NP-difíciles?** Porque selecciona alternativas al azar y no aprovecha por sí solo la estructura del problema; en espacios enormes, la probabilidad de probar justo una ruta óptima puede ser muy baja. Para obtener mejores resultados prácticos se usan heurísticas que guían y mejoran la búsqueda. El

caso ORION ilustra esto: combina técnicas de optimización adaptadas a sus restricciones, y no está documentado como una búsqueda Monte Carlo pura.

3. Conclusiones

Los ejercicios muestran que no existe un único método adecuado para todos los tamaños. La fuerza bruta de los primeros ejemplos da una respuesta óptima y comprobable, pero el número de casos crece muy rápido. Monte Carlo reduce el trabajo al probar solo algunas rutas: en el TSP y en la asignación de tareas puede encontrar buenas soluciones con rapidez, aunque no asegura el óptimo.

La comparación por fuerza bruta del ejercicio de producción permitió confirmar el resultado aleatorio porque solo había 64 asignaciones. En cambio, la tabla de crecimiento y el caso de UPS ORION muestran por qué esa estrategia no se traslada a problemas grandes: una ruta real puede incluir entre 120 y 160 paradas, además de horarios, tráfico y criterios de seguridad. Allí se necesitan heurísticas que aprovechen la estructura del problema y produzcan rutas útiles dentro del tiempo disponible.

En conjunto, el laboratorio evidencia el equilibrio central del diseño de algoritmos: los métodos exactos ofrecen garantía cuando el espacio es pequeño; los heurísticos sacrifican esa garantía para abordar instancias grandes. También enseña que modelar un caso real requiere incluir restricciones prácticas que no aparecen en la versión académica más simple del problema.

Referencias

Clevenger, Seth (9 de ago. de 2016). *UPS Routing Program ORION Helps Drivers Trim Miles, Reduce Costs*. Transport Topics. URL: https://www.ttnews.com/articles/ups-routing-program-orion-helps-drivers-trim-miles-reduce-costs.

Garey, Michael R. y David S. Johnson (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco, CA: W. H. Freeman. ISBN: 978-0-7167-1045-5.

Held, Michael y Richard M. Karp (1962). «A Dynamic Programming Approach to Sequencing Problems». En: *Journal of the Society for Industrial and Applied Mathematics* 10.1, págs. 196-210. DOI: 10.1137/0110015.

Holland, Chuck et al. (2017). «UPS Optimizes Delivery Routes». En: *Interfaces* 47.1, págs. 8-23. DOI: 10.1287/inte.2016.0875.

Ingber, Lester (1993). «Simulated Annealing: Practice versus Theory». En: *Mathematical and Computer Modelling* 18.11, págs. 29-57. DOI: 10.1016/0895-7177(93)90204-C.

Lemaréchal, Claude (2001). «Lagrangian Relaxation». En: *Computational Combinatorial Optimization: Optimal or Provably Near-Optimal Solutions*. Ed. por Michael Jünger y Denis Naddef. Vol. 2241. Lecture Notes in Computer Science. Berlin: Springer, págs. 112-156. DOI: 10.1007/3-540-45586-8_4.

Leonard, Matt (11 de jun. de 2021). *UPS adds dynamic routing to ORION, saving 2-4 miles per driver*. Supply Chain Dive. URL: https://www.supplychaindive.com/news/ups-orion-route-planning-analytics-data-logistics/601673/.

Lin, Shen y Brian W. Kernighan (1973). «An Effective Heuristic Algorithm for the Traveling-Salesman Problem». En: *Operations Research* 21.2, págs. 498-516. DOI: 10.1287/opre.21.2.498.

Ropke, Stefan y David Pisinger (2006). «An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows». En: *Transportation Science* 40.4, págs. 455-472. DOI: 10.1287/trsc.1050.0135.

UPS (3 de mar. de 2015). *UPS Accelerates Use of Routing Optimization Software to Reduce 100 Million Miles Driven*. Comunicado de prensa. URL: https://www.globenewswire.com/de/news-release/2015/03/02/711508/30428/en/UPS-Accelerates-Use-of-Routing-Optimization-Software-to-Reduce-100-Million-Miles-Driven.html.

UPS (29 de ene. de 2020). *UPS To Enhance ORION With Continuous Delivery Route Optimization*. Comunicado de prensa. URL: `https://about.ups.com/ae/en/newsroom/press-releases/innovation-driven/ups-to-enhance-orion-with-continuous-delivery-route-optimization.html`.